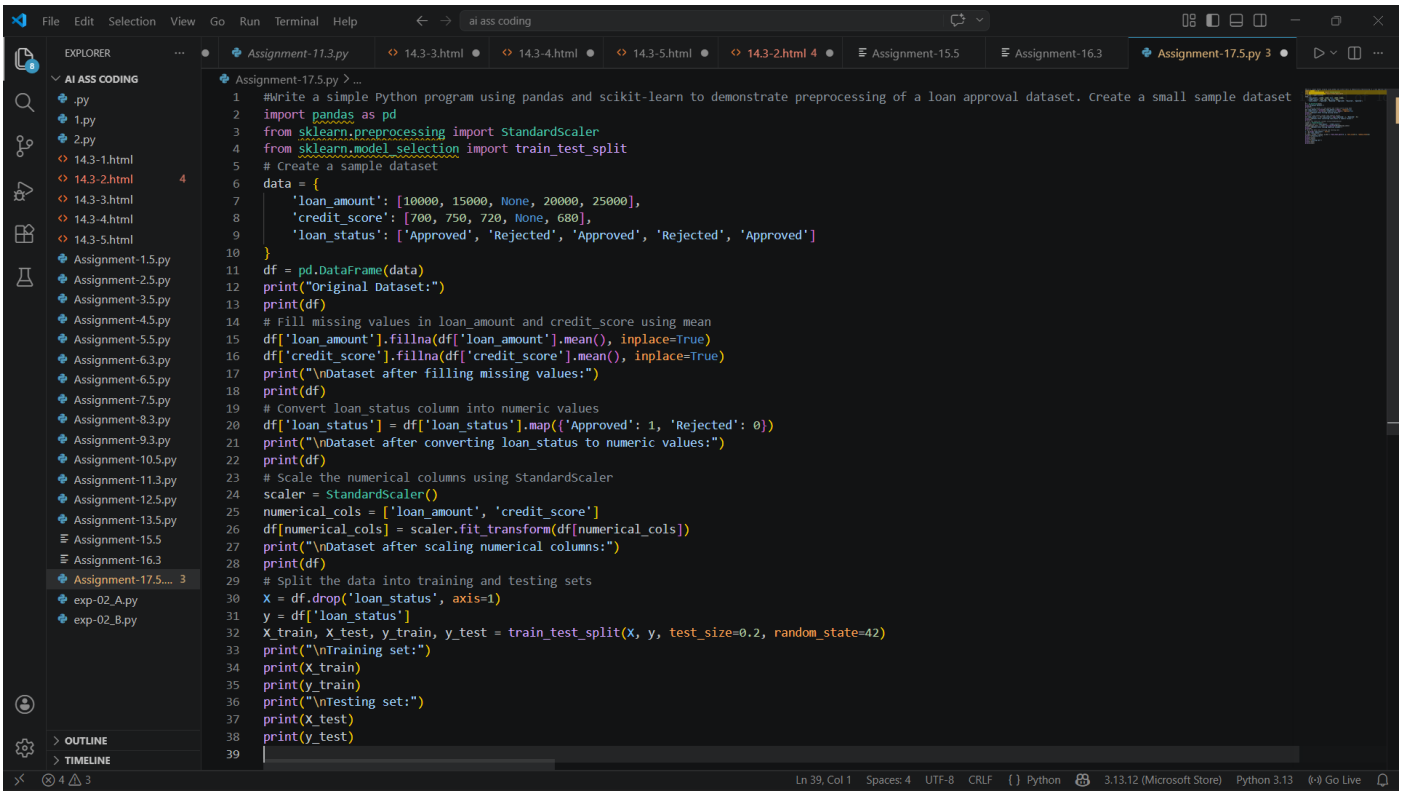
ASSIGNMENT 17.5

2303A51356
POOJITHA EDDE
Batch 20

Task 1 –Loan Approval Dataset Cleaning

PROMPT:
Write a simple Python program using pandas and scikit-learn to demonstrate preprocessing of a loan approval dataset. Create a small sample dataset instead of loading from a CSV file. Fill missing values in loan_amount and credit_score using mean or median. Convert the loan_status column into numeric values (Approved = 1, Rejected = 0). Scale the numerical columns using StandardScaler, and then split the data into training and testing sets using train_test_split. Print outputs after each step to clearly show the changes.

CODE:



```python
#Write a simple Python program using pandas and scikit-learn to demonstrate preprocessing of a loan approval dataset. Create a small sample dataset
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
# Create a sample dataset
data = {
    'loan_amount': [10000, 15000, None, 20000, 25000],
    'credit_score': [700, 750, 720, None, 680],
    'loan_status': ['Approved', 'Rejected', 'Approved', 'Rejected', 'Approved']
}
df = pd.DataFrame(data)
print("Original Dataset:")
print(df)
# Fill missing values in loan_amount and credit_score using mean
df['loan_amount'].fillna(df['loan_amount'].mean(), inplace=True)
df['credit_score'].fillna(df['credit_score'].mean(), inplace=True)
print("\nDataset after filling missing values:")
print(df)
# Convert loan_status column into numeric values
df['loan_status'] = df['loan_status'].map({'Approved': 1, 'Rejected': 0})
print("\nDataset after converting loan_status to numeric values:")
print(df)
# Scale the numerical columns using StandardScaler
scaler = StandardScaler()
numerical_cols = ['loan_amount', 'credit_score']
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])
print("\nDataset after scaling numerical columns:")
print(df)
# Split the data into training and testing sets
X = df.drop('loan_status', axis=1)
y = df['loan_status']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("\nTraining set:")
print(X_train)
print(y_train)
print("\nTesting set:")
print(X_test)
print(y_test)
```

OUTPUT:

```
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding> & C:/Users/pooji/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/pooji/OneDrive/Desktop/ai ass coding/Assignment-17
.5.py"
Original Dataset:
   loan_amount  credit_score loan_status
0      10000.0         700.0    Approved
1      15000.0         750.0    Rejected
2          NaN         720.0    Approved
3      20000.0           NaN    Rejected
4      25000.0         680.0    Approved

After Handling Missing Values:
   loan_amount  credit_score loan_status
0      10000.0         700.0    Approved
1      15000.0         750.0    Rejected
2      17500.0         720.0    Approved
3      20000.0         712.5    Rejected
4      25000.0         680.0    Approved

After Encoding loan_status:
   loan_amount  credit_score  loan_status
0      10000.0         700.0            1
1      15000.0         750.0            0
2      17500.0         720.0            1
3      20000.0         712.5            0
4      25000.0         680.0            1

After Scaling:
   loan_amount  credit_score  loan_status
0         -1.5     -0.540422            1
1         -0.5      1.621266            0
2          0.0      0.324253            1
3          0.5      0.000000            0
4          1.5     -1.405098            1

X_train:
   loan_amount  credit_score
4          1.5     -1.405098
2          0.0      0.324253
0         -1.5     -0.540422
3          0.5      0.000000

X_test:
   loan_amount  credit_score
```

```
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding> & C:/Users/pooji/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/pooji/OneDrive/Desktop/ai ass coding/Assignment-17
.5.py"

After Scaling:
   loan_amount  credit_score  loan_status
0         -1.5     -0.540422            1
1         -0.5      1.621266            0
2          0.0      0.324253            1
3          0.5      0.000000            0
4          1.5     -1.405098            1

X_train:
   loan_amount  credit_score
4          1.5     -1.405098
2          0.0      0.324253
0         -1.5     -0.540422
3          0.5      0.000000

X_test:
   loan_amount  credit_score
1         -0.5      1.621266

y_train:
4    1
2    1
0    1
3    0
Name: loan_status, dtype: int64

y_test:
1    0
Name: loan_status, dtype: int64
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding>
```

Task 2 – Social Media Profiles

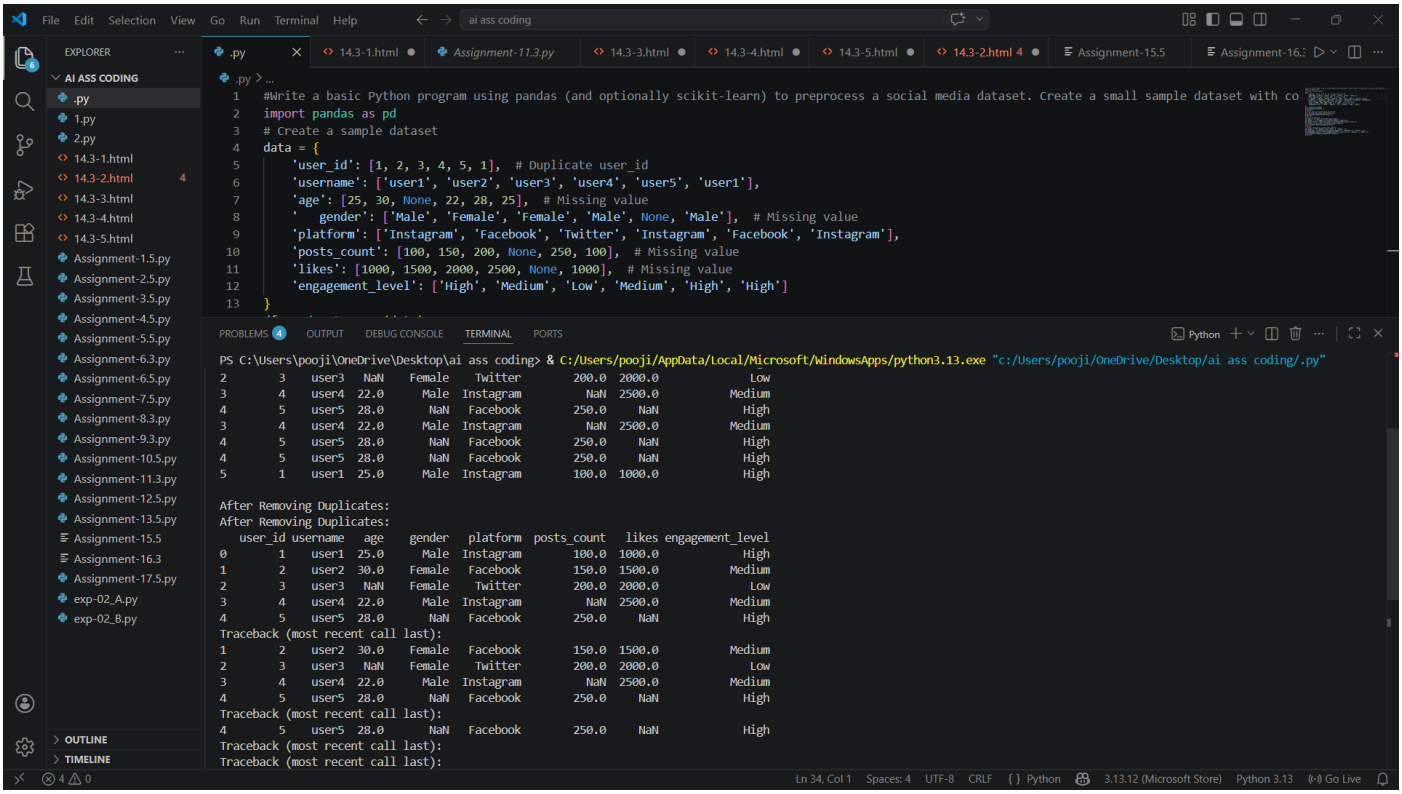
PROMPT:

Write a basic Python program using pandas (and optionally scikit-learn) to preprocess a social media dataset. Create a small sample dataset with columns like user_id, username, age, gender, platform, posts_count, likes, and engagement_level. Remove duplicate users based on user_id, fill missing values (use mean/median for numbers and mode for categories), and convert categorical columns (gender, platform, engagement_level) into numbers. Print the dataset before and after each step to show the changes clearly.

CODE:

```python
#Write a basic Python program using pandas (and optionally scikit-learn) to preprocess a social media dataset. Create a small sample dataset with co
import pandas as pd
# Create a sample dataset
data = {
    'user_id': [1, 2, 3, 4, 5, 1],  # Duplicate user_id
    'username': ['user1', 'user2', 'user3', 'user4', 'user5', 'user1'],
    'age': [25, 30, None, 22, 28, 25],  # Missing value
    '   gender': ['Male', 'Female', 'Female', 'Male', None, 'Male'],  # Missing value
    'platform': ['Instagram', 'Facebook', 'Twitter', 'Instagram', 'Facebook', 'Instagram'],
    'posts_count': [100, 150, 200, None, 250, 100],  # Missing value
    'likes': [1000, 1500, 2000, 2500, None, 1000],  # Missing value
    'engagement_level': ['High', 'Medium', 'Low', 'Medium', 'High', 'High']
}
df = pd.DataFrame(data)
print("Original Dataset:")
print(df)
# Remove duplicate users based on user_id
df = df.drop_duplicates(subset='user_id')
print("\nAfter Removing Duplicates:")
print(df)
# Fill missing values
df['age'] = df['age'].fillna(df['age'].mean())
df['gender'] = df['gender'].fillna(df['gender'].mode()[0])
df['posts_count'] = df['posts_count'].fillna(df['posts_count'].mean ())
df['likes'] = df['likes'].fillna(df['likes'].mean())
print("\nAfter Handling Missing Values:")
print(df)
# Convert categorical columns into numbers
df['gender'] = df['gender'].map({'Male': 0, 'Female': 1})
df['platform'] = df['platform'].map({'Instagram': 0, 'Facebook': 1, 'Twitter': 2})
df['engagement_level'] = df['engagement_level'].map({'Low': 0, 'Medium': 1, 'High': 2})
print("\nAfter Encoding Categorical Columns:")
print(df)
```

OUTPUT:



Task 3 – Weather Dataset Feature Engineering

PROMPT:

Write a simple Python program using pandas and scikit-learn to preprocess a weather dataset. Create a small sample dataset with columns like date, temperature, humidity, and rainfall. Convert the date column to datetime format, fill missing values in temperature and humidity using mean or median, and create new features like season and week number from the date. Scale the rainfall column using a scaler like MinMaxScaler. Print the dataset before and after each step to clearly show the changes.

CODE:

```python
#Write a simple Python program using pandas and scikit-learn to preprocess a weather dataset. Create a small sample dataset with columns like date,
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
# Create a sample dataset
data = {
    'date': ['2024-01-01', '2024-02-01', '2024-03-01', '2024-04-01', '2024-05-01'],
    'temperature': [30, 25, None, 20, 15],  # Missing value
    'humidity': [80, None, 70, 60, 50],  # Missing value
    'rainfall': [5, 10, 0, 15, 20]
}
df = pd.DataFrame(data)
print("Original Dataset:")
print(df)
# Convert the date column to datetime format
df['date'] = pd.to_datetime(df['date'])
print("\nAfter Converting Date to Datetime:")
print(df)
# Fill missing values in temperature and humidity using mean
df['temperature'] = df['temperature'].fillna(df['temperature'].mean())
df['humidity'] = df['humidity'].fillna(df['humidity'].mean())
print("\nAfter Handling Missing Values:")
print(df)
# Create new features like season and week number from the date
df['season'] = df['date'].dt.month % 12 // 3
1  # 1: Winter, 2: Spring, 3: Summer, 4: Fall
df['week_number'] = df['date'].dt.isocalendar().week
print("\nAfter Creating New Features:")
print(df)
# Scale the rainfall column using MinMaxScaler
scaler = MinMaxScaler()
df['rainfall'] = scaler.fit_transform(df[['rainfall']])
print("\nAfter Scaling Rainfall:")
print(df)
```

OUTPUT:

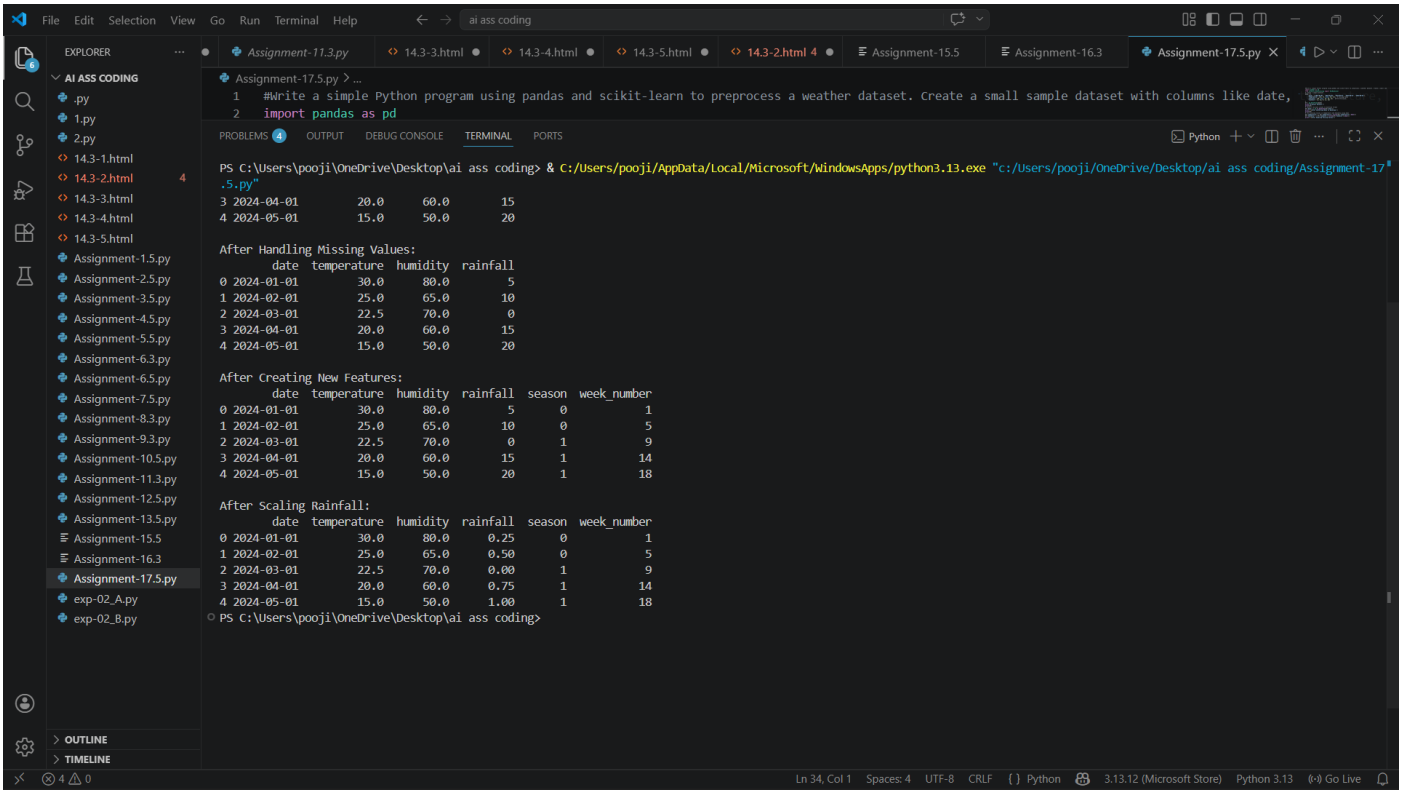
```
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding> & C:/Users/pooji/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/pooji/OneDrive/Desktop/ai ass coding/Assignment-17
.5.py"
Original Dataset:
         date  temperature  humidity  rainfall
0  2024-01-01         30.0      80.0         5
1  2024-02-01         25.0       NaN        10
2  2024-03-01          NaN      70.0         0
3  2024-04-01         20.0      60.0        15
4  2024-05-01         15.0      50.0        20

After Converting Date to Datetime:
        date  temperature  humidity  rainfall
0 2024-01-01         30.0      80.0         5
1 2024-02-01         25.0       NaN        10
2 2024-03-01          NaN      70.0         0
3 2024-04-01         20.0      60.0        15
4 2024-05-01         15.0      50.0        20

After Handling Missing Values:
        date  temperature  humidity  rainfall
0 2024-01-01         30.0      80.0         5
1 2024-02-01         25.0      65.0        10
2 2024-03-01         22.5      70.0         0
3 2024-04-01         20.0      60.0        15
4 2024-05-01         15.0      50.0        20

After Creating New Features:
        date  temperature  humidity  rainfall  season  week_number
0 2024-01-01         30.0      80.0         5       0            1
1 2024-02-01         25.0      65.0        10       0            5
2 2024-03-01         22.5      70.0         0       1            9
3 2024-04-01         20.0      60.0        15       1           14
4 2024-05-01         15.0      50.0        20       1           18

After Scaling Rainfall:
        date  temperature  humidity  rainfall  season  week_number
0 2024-01-01         30.0      80.0      0.25       0            1
1 2024-02-01         25.0      65.0      0.50       0            5
```



Task 4 – Movie Ratings Dataset Preparation

PROMPT:

Write a simple Python script using pandas and scikit-learn to clean a movie dataset. Create a small sample dataset with columns such as movie_id, title, genre, language, and rating. Remove duplicate records using movie_id, handle missing values (fill rating with mean and genre with the most frequent value), and encode categorical columns like genre and language into numbers. Finally, normalize the rating values using MinMaxScaler. Display the dataset at each step to show how the data changes after preprocessing.

Code:

```python
import pandas
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import LabelEncoder
# Create a sample dataset
data = {
    'movie_id': [1, 2, 3, 4, 5, 5],  # Duplicate movie_id
    'title': ['Movie A', 'Movie B', 'Movie C', 'Movie D', 'Movie E', 'Movie E'],
    'genre': ['Action', 'Comedy', None, 'Action', 'Comedy', 'Comedy'],  # Missing genre
    'language': ['English', 'French', 'English', None, 'French', 'French'],  # Missing language
    'rating': [8.5, 7.0, None, 9.0, 6.5, 6.5]  # Missing rating
}
# Create a DataFrame
df = pd.DataFrame(data)
print("Original Dataset:")
print(df)
# Remove duplicate records using movie_id
df = df.drop_duplicates(subset='movie_id')
print("\nDataset after removing duplicates:")
print(df)
# Handle missing values
# Fill missing rating with mean
mean_rating = df['rating'].mean()
df['rating'] = df['rating'].fillna(mean_rating)
# Fill missing genre with the most frequent value
most_frequent_genre = df['genre'].mode()[0]
df['genre'] = df['genre'].fillna(most_frequent_genre)
# Fill missing language with the most frequent value
most_frequent_language = df['language'].mode()[0]
df['language'] = df['language'].fillna(most_frequent_language)
print("\nDataset after handling missing values:")
print(df)
# Encode categorical columns
label_encoder_genre = LabelEncoder()
label_encoder_language = LabelEncoder()
df['genre'] = label_encoder_genre.fit_transform(df['genre'])
df['language'] = label_encoder_language.fit_transform(df['language'])
print("\nDataset after encoding categorical columns:")
print(df)
# Normalize the rating values using MinMaxScaler
scaler = MinMaxScaler()
df['rating'] = scaler.fit_transform(df[['rating']])
print("\nDataset after normalizing rating values:")
print(df)
```

OUTPUT:

```
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding> & C:/Users/pooji/AppData/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/pooji/OneDrive/Desktop/ai ass coding/.py"
Original Dataset:
   movie_id    title   genre language  rating
0         1  Movie A  Action  English     8.5
1         2  Movie B  Comedy   French     7.0
2         3  Movie C     NaN  English     NaN
3         4  Movie D  Action      NaN     9.0
4         5  Movie E  Comedy   French     6.5
5         5  Movie E  Comedy   French     6.5

Dataset after removing duplicates:
   movie_id    title   genre language  rating
0         1  Movie A  Action  English     8.5
1         2  Movie B  Comedy   French     7.0
2         3  Movie C     NaN  English     NaN
3         4  Movie D  Action      NaN     9.0
4         5  Movie E  Comedy   French     6.5

Dataset after handling missing values:
   movie_id    title   genre language  rating
0         1  Movie A  Action  English    8.50
1         2  Movie B  Comedy   French    7.00
2         3  Movie C  Action  English    7.75
3         4  Movie D  Action  English    9.00
4         5  Movie E  Comedy   French    6.50

Dataset after encoding categorical columns:
   movie_id    title  genre  language  rating
0         1  Movie A      0         0    8.50
1         2  Movie B      1         1    7.00
2         3  Movie C      0         0    7.75
3         4  Movie D      0         0    9.00
4         5  Movie E      1         1    6.50

Dataset after normalizing rating values:
   movie_id    title  genre  language  rating
0         1  Movie A      0         0     0.8
1         2  Movie B      1         1     0.2
2         3  Movie C      0         0     0.5
3         4  Movie D      0         0     1.0
4         5  Movie E      1         1     0.0
PS C:\Users\pooji\OneDrive\Desktop\ai ass coding>
```